

TECHNICAL DOCUMENTATION

Architecture & Implementation Reference

A ground-truth walkthrough of the Taskflow codebase — architecture, layering, request flow, authentication, and database design, derived entirely from the source in this repository.

Angular 21 (standalone + signals)

Spring Boot 3.4.4 / Java 21

PostgreSQL 16

JWT (jjwt 0.12.6)

Contents

1. Project Overview

2. Technology Stack

3. Software Architecture

4. Folder & Package Structure

5. Major Classes & Responsibilities

6. How the Frontend, Backend, and Database Communicate

7. Authentication Flow (JWT)

8. Database Structure

9. API Request Flow

10. API Reference

Every statement in this document is traceable to a specific file in the codebase (backend Java sources under `backend/src/main/java/com/taskmanager`, frontend TypeScript under `frontend/src/app`, and `database/schema.sql`). Where the implementation has a notable gap or an unused code path, it is called out explicitly in a callout box rather than glossed over.

SECTION 1

Project Overview

Taskflow is a full-stack, multi-user task management system. Each user registers an account, signs in, and manages a personal list of tasks — creating, editing, searching, filtering, sorting, and deleting them. Every task belongs to exactly one user, and the system enforces that a user can only ever see or modify their own tasks.

The system is built as two independently deployable applications that communicate over HTTP:

- A **backend REST API** (Spring Boot, Java 21) that owns all business logic, security, and persistence, listening on `localhost:8080`.
- A **frontend single-page application** (Angular 21) that renders the UI and talks to the API exclusively over JSON, served in development on `localhost:4200`.

Both sit in front of a **PostgreSQL 16** database that the backend accesses through Hibernate/JPA — the frontend never talks to the database directly.

Core capabilities (verified in code)

Authentication

Register and log in with email/password. Passwords are hashed with BCrypt (`SecurityConfig.passwordEncoder()`). Sessions are stateless — every request re-authenticates via a JWT bearer token (`JwtService`, `JwtAuthenticationFilter`).

Task CRUD

Full create/read/update/delete on tasks, scoped per owner at the repository layer (`TaskRepository.findByIdAndUserId`), so ownership can't be bypassed by guessing another user's task id.

Search, filter, sort, paginate

Keyword search over title/description, filter by status and priority, sortable by a whitelisted set of fields, paginated — all composed dynamically with JPA Specifications (`TaskSpecifications`) inside `TaskService.search()`.

Consistent error handling

Every exception — validation, not-found, conflict, bad credentials, access-denied, or unexpected — is funnelled through one `@RestControllerAdvice` (`GlobalExceptionHandler`) into a single JSON shape (`ApiError`).

Deployment shape: this is a classic decoupled client-server system, not a monolith with server-rendered pages. There is no HTML templating on the backend (no Thymeleaf/JSP) — the Spring Boot application is a pure JSON API, and the entire user interface lives in the Angular application.

SECTION 2

Technology Stack

Backend — backend/pom.xml

Component	Technology	Role
Language / runtime	Java 21	Language level pinned in <code><java.version></code>
Framework	Spring Boot 3.4.4 (parent BOM)	Application framework, auto-configuration, embedded Tomcat
Web	spring-boot-starter-web	REST controllers, embedded servlet container
Persistence	spring-boot-starter-data-jpa (Hibernate)	ORM, repositories, Specification-based queries
Validation	spring-boot-starter-validation	Bean Validation annotations on request DTOs
Security	spring-boot-starter-security	Filter chain, password encoding, authentication manager
JWT	io.jsonwebtoken (jjwt) 0.12.6 — api / impl / jackson	Signing and parsing JSON Web Tokens
Database driver	org.postgresql:postgresql (runtime scope)	JDBC driver for PostgreSQL
Build	Maven + spring-boot-maven-plugin	Build/packaging, <code>mvn spring-boot:run</code>
Testing	spring-boot-starter-test, spring-security-test	Declared as dependencies (no test sources currently present in <code>src/test</code>)

Frontend — frontend/package.json

Component	Technology	Role
Framework	Angular ^21.0.0	Standalone components (no <code>NgModule</code> s anywhere in the app)
Language	TypeScript 5.9.2	Strict-mode application code
Reactivity	Angular Signals (<code>signal</code> , <code>computed</code>)	Local component/service state — no NgRx or global store library is used
HTTP	RxJS 7.8 + <code>HttpClient</code>	Observable-based API calls with functional interceptors
Routing	@angular/router	Lazy-loaded standalone routes with functional guards
Build	Angular CLI / @angular-devkit/build-angular	<code>ng serve</code> / <code>ng build</code>

Database

Component	Detail
Engine	PostgreSQL 16 (per README prerequisites)
Schema management	<code>spring.jpa.hibernate.ddl-auto: update</code> — Hibernate creates/updates tables from the <code>@Entity</code> classes on every boot. <code>database/schema.sql</code> is a hand-maintained equivalent DDL script for manual/production provisioning.

Tables `users`, `tasks` (see Section 8)

Configuration — `application.yml`

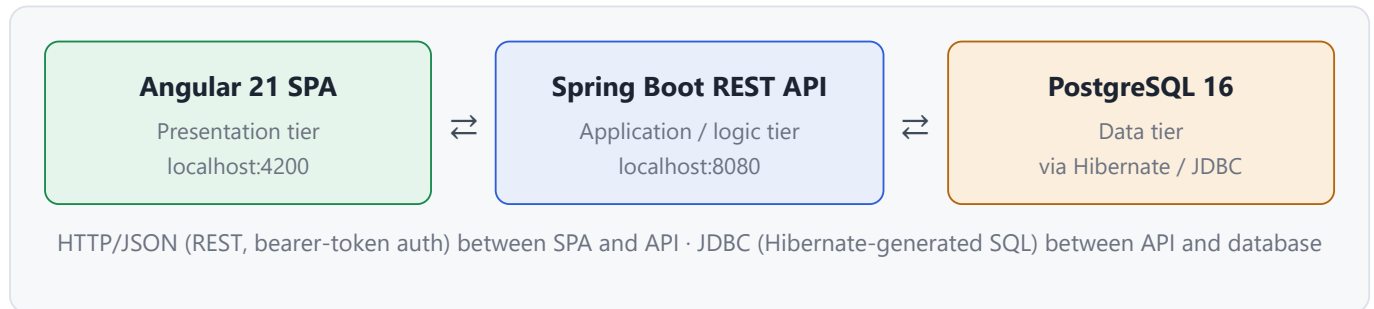
All environment-specific values are externalized with sensible dev defaults and can be overridden by environment variables at runtime:

Setting	Env var	Default
Server port	<code>SERVER_PORT</code>	8080
Datasource URL / user / password	<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code>	localhost / 5432 / task_manager / postgres / postgres
JWT signing secret	<code>APP_JWT_SECRET</code>	Base64 dev secret embedded in <code>application.yml</code> (must be overridden in production)
JWT expiration	<code>APP_JWT_EXPIRATION_MS</code>	86400000 ms (24 hours)
Allowed CORS origin(s)	<code>APP_CORS_ORIGINS</code>	http://localhost:4200

SECTION 3

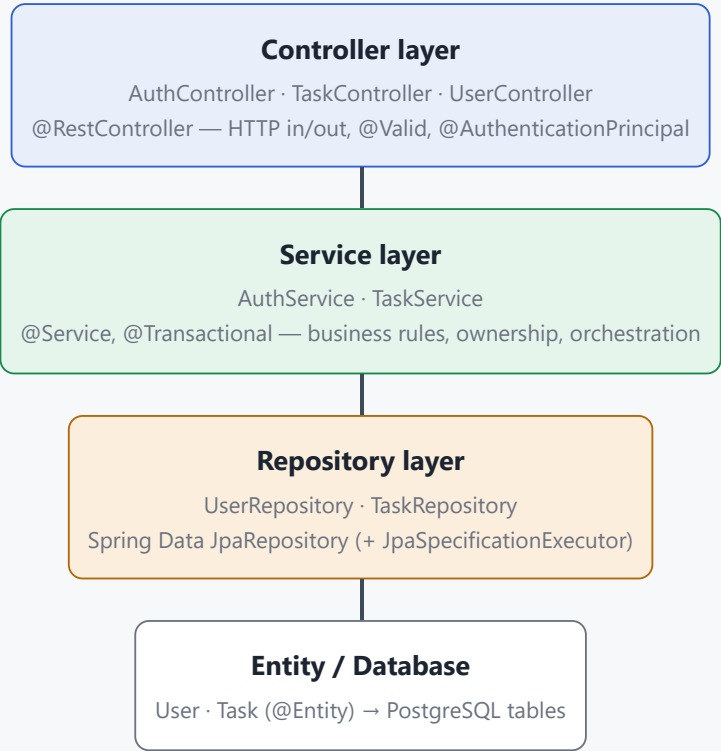
Software Architecture

Taskflow is a **three-tier client-server system**, and the backend tier is internally organized as a classic **Layered Architecture** — the same Controller → Service → Repository → Entity structure Spring Data applications are built around. It is **not** an MVC application in the traditional server-rendered sense, and it does not use Hexagonal/Clean Architecture module boundaries (no separate `domain` / `application` / `infrastructure` modules, no explicit port interfaces distinct from Spring Data repositories).



Why "Layered Architecture", not MVC

- **No server-side View layer exists.** There are no Thymeleaf/JSP templates anywhere in `backend/src` — every controller returns a `ResponseEntity<DTO>` serialized straight to JSON. The "View" responsibility belongs entirely to the separately deployed Angular app, so calling the backend "MVC" would misdescribe it.
- **Strict one-directional dependency flow.** Every controller (`AuthController` , `TaskController` , `UserController`) depends only on a service; every service (`AuthService` , `TaskService`) depends only on a repository; every repository depends only on a JPA entity. There is no reverse dependency (a repository never calls a service, a service never touches `HttpServletRequest`) — the defining trait of a layered architecture.
- **Clear responsibility split per layer**, evidenced directly in the code: controllers contain zero business logic (they validate input via `@Valid` and translate to HTTP status codes); services contain all business rules (ownership checks, uniqueness checks, sort-field whitelisting); repositories contain no logic at all beyond declarative Spring Data query derivation and `Specification` composition.



Backend layering. Two cross-cutting concerns wrap every request outside this vertical stack: the Spring Security filter chain (runs before the controller layer) and GlobalExceptionHandler (intercepts anything thrown by any layer).

Package-by-feature, layered-by-class

At the top level, backend code is grouped **by feature/domain** (`auth` , `task` , `user` , `security` , `config` , `exception`) rather than by technical role. Inside each feature package, however, classes are still cleanly separated by role — e.g. `task/` contains `TaskController` , `TaskService` , `TaskRepository` , and a nested `dto/` package side by side. This is a common, deliberate hybrid: it keeps a feature's Controller/Service/Repository/DTOs discoverable together while still preserving the layered dependency direction.

Cross-cutting layers

Security filter chain

`SecurityConfig` registers `JwtAuthenticationFilter` ahead of Spring Security's standard `UsernamePasswordAuthenticationFilter` , with `SessionCreationPolicy.STATELESS` (no `HttpSession` is ever created). This intercepts every request before it reaches a controller.

Centralized error translation

`GlobalExceptionHandler` (`@RestControllerAdvice`) converts every exception type raised anywhere — validation, not-found, conflict, bad credentials, access-denied, or unhandled — into one consistent `ApiError` JSON shape with the correct HTTP status.

DTO boundary

Request/response records (`TaskRequest` / `TaskResponse` , `UserResponse` , `AuthResponse` , etc.) decouple the public API

Auditing

`@EnableJpaAuditing` on `TaskManagerApplication` , combined with `@CreatedDate` / `@LastModifiedDate` on `Task` and

contract from the JPA entities — e.g. `UserResponse` deliberately omits the password field that `User` carries.

`@CreateDate` on `User`, auto-stamps timestamps without any explicit service code.

Frontend architecture

The Angular app follows the framework's recommended **feature-based folder convention**, expressed with modern standalone APIs rather than classic `NgModule`s:

- `core/` — cross-cutting concerns shared by the whole app: `models` (TypeScript types mirroring backend DTOs), `services` (HTTP clients), `guards` (route protection), `interceptors` (cross-cutting HTTP behavior).
- `features/` — page-level, domain-specific components: `auth` (login, register), `dashboard` (task list + task form).
- `shared/` — reusable presentational pieces used across features, currently just `navbar`.

State is handled with **Angular Signals** (`signal()` , `computed()`) local to each service/component — there is no NgRx, Akita, or other global store. Route guards (`authGuard` , `guestGuard`) and HTTP interceptors (`authInterceptor` , `errorInterceptor`) are written as plain functions (Angular's functional guard/interceptor API), not classes.

SECTION 4

Folder & Package Structure

Backend — backend/src/main/java/com/taskmanager

```
com.taskmanager
├─ TaskManagerApplication.java  Spring Boot entry point (@EnableJpaAuditing)
├─ auth/                        registration & login
│   ├── AuthController.java
│   ├── AuthService.java
│   └─ dto/  LoginRequest · RegisterRequest · AuthResponse · UserResponse
├─ user/                        user identity & profile
│   ├── User.java              @Entity implementing Spring Security UserDetails
│   ├── Role.java              enum: ROLE_USER, ROLE_ADMIN
│   ├── UserRepository.java
│   └─ UserController.java     GET /api/users/me
├─ task/                        task domain (largest package)
│   ├── Task.java              @Entity, ManyToOne owner
│   ├── Priority.java · TaskStatus.java  enums
│   ├── TaskRepository.java    JpaRepository + JpaSpecificationExecutor
│   ├── TaskSpecifications.java dynamic query building blocks
│   ├── TaskService.java
│   ├── TaskController.java
│   └─ dto/  TaskRequest · TaskResponse · PageResponse<T>
├─ security/                    JWT + Spring Security plumbing
│   ├── JwtService.java
│   ├── JwtAuthenticationFilter.java
│   ├── JwtAuthenticationEntryPoint.java
│   └─ AppUserDetailsService.java
├─ config/
│   └─ SecurityConfig.java     filter chain, CORS, password encoder, auth beans
└─ exception/                  cross-cutting error handling
    ├── GlobalExceptionHandler.java
    ├── ApiError.java
    ├── ResourceNotFoundException.java
    └─ EmailAlreadyExistsException.java
```

Frontend — frontend/src/app

```
app/
├─ app.component.ts            root shell — shows <app-navbar> when authenticated
├─ app.config.ts               providers: router, HttpClient + interceptors
├─ app.routes.ts               route table, lazy-loaded standalone components
├─ core/
│   ├── models/  task.model.ts · user.model.ts
│   ├── services/ auth.service.ts · task.service.ts
│   ├── guards/  auth.guard.ts  authGuard, guestGuard
│   └─ interceptors/ auth.interceptor.ts · error.interceptor.ts
├─ features/
│   └─ auth/
│       ├── login/  login.component.ts
│       └─ register/ register.component.ts
```

```
|   └─ dashboard/
|       └─ dashboard.component.ts    search/filter/sort/paginate, task grid
|       └─ task-form.component.ts    create/edit modal
└─ shared/navbar/ navbar.component.ts
```

Repository root

```
task-management-system/
├─ backend/      Spring Boot Maven project
├─ frontend/     Angular CLI project
├─ database/     schema.sql  reference DDL (tables are also auto-created by Hibernate)
├─ docs/         existing design notes (00-07)
└─ README.md
```

SECTION 5

Major Classes & Responsibilities

Bootstrap

TaskManagerApplication `com.taskmanager`

The Spring Boot entry point. `@SpringBootApplication` triggers component scanning and auto-configuration; `@EnableJpaAuditing` activates the `@CreatedDate` / `@LastModifiedDate` annotations used on `User` and `Task`.

Package: `config`

SecurityConfig

Defines the entire security posture in one place: builds the `SecurityFilterChain` (CORS enabled, CSRF disabled — safe here because the API is stateless and uses bearer tokens rather than cookies), whitelists `/api/auth/**` and `/actuator/health` as public, requires authentication for everything else, sets `SessionCreationPolicy.STATELESS`, wires the custom `JwtAuthenticationEntryPoint` for unauthenticated responses, and inserts `JwtAuthenticationFilter` before Spring's default `UsernamePasswordAuthenticationFilter`. Also exposes the `PasswordEncoder` (BCrypt), `AuthenticationProvider` (`DaoAuthenticationProvider`), and `AuthenticationManager` beans used by `AuthService`.

Package: `security`

JwtService

Issues and verifies JWTs using the jjwt library. The signing key is derived from `app.jwt.secret` (HMAC-SHA); tokens carry the user's **email** as the subject, an `issuedAt`, and an `expiration` computed from `app.jwt.expiration-ms` (24h by default). `isTokenValid()` checks both subject match and non-expiry.

JwtAuthenticationFilter

A `OncePerRequestFilter` that runs on every request. Reads the `Authorization` header, and if it's a `Bearer` token, extracts the email, loads the corresponding `UserDetails`, and — if the token is valid — manually populates a `UsernamePasswordAuthenticationToken` into `SecurityContextHolder`. Any exception during parsing is caught and the security context is simply cleared, deferring the actual 401 response to the entry point rather than throwing from the filter.

JwtAuthenticationEntryPoint

Fires whenever Spring Security rejects an unauthenticated request to a protected endpoint. Writes a JSON `ApiError` body (status 401) directly to the response, so unauthenticated failures have the exact same shape as every other API error.

AppUserDetailsService

The Spring Security `UserDetailsService` implementation. Looks a user up **by email** via `UserRepository.findByEmail` — email is the "username" for authentication purposes throughout this system.

Package: `auth`

AuthController

`POST /api/auth/register` and `POST /api/auth/login`. Both endpoints are unauthenticated (permitted in `SecurityConfig`). The controller is a thin pass-through to `AuthService`, returning `201 Created` for registration and `200 OK` for login.

AuthService

`register()`: checks `existsByEmail`, throws `EmailAlreadyExistsException` if taken, otherwise BCrypt-encodes the password, builds a `User` defaulted to `ROLE_USER`, persists it, and issues a JWT. `login()`: delegates credential verification entirely to Spring Security's `AuthenticationManager` (which throws `BadCredentialsException` on failure), then issues a fresh JWT for the authenticated principal. Both methods return an `AuthResponse`.

DTOs — LoginRequest, RegisterRequest, AuthResponse, UserResponse

Java records with Bean Validation constraints (`@NotBlank`, `@Email`, `@Size`). `UserResponse.from(User)` and `AuthResponse.of(token, user)` are static factories that project the entity into a client-safe shape — notably, `UserResponse` never carries the password hash.

Package: `user`

User `@Entity` implements `UserDetails`

Dual-purpose class: it is both the JPA entity mapped to the `users` table **and** the Spring Security principal. Its `getUsername()` returns the email; `getAuthorities()` maps the single `Role` enum value to one `SimpleGrantedAuthority`. Uses a nested static `Builder` rather than a fluent-setter or Lombok-generated builder.

Role `enum`

`ROLE_USER`, `ROLE_ADMIN`. New accounts always get `ROLE_USER` (see `AuthService.register`).

Observation: `ROLE_ADMIN` is declared but never checked anywhere. `@EnableMethodSecurity` is turned on in `SecurityConfig`, yet no controller or service uses `@PreAuthorize`, `hasRole`, or `hasAuthority`. Every authenticated user — regardless of role — has identical access to all task endpoints. There is also no admin-only endpoint anywhere in the codebase.

UserRepository / UserController

`UserRepository` adds `findByEmail` and `existsByEmail` to the standard `JpaRepository`. `UserController` exposes a single endpoint, `GET /api/users/me`, which resolves the caller straight from `@AuthenticationPrincipal` — no explicit lookup needed since the JWT filter already placed the full `User` entity in the security context.

Package: `task`

Task `@Entity`

Fields: `title`, `description`, `priority`, `status`, `dueDate`, a `@ManyToOne` lazy-fetched `user` (the owner, non-nullable), plus auditing timestamps. Indexed on `user_id`, `status`, `priority`, and `due_date` — exactly the columns the search/filter feature queries against.

TaskRepository

Extends both `JpaRepository<Task, Long>` and `JpaSpecificationExecutor<Task>`. The `JpaSpecificationExecutor` is what allows `TaskService` to compose dynamic, type-safe `WHERE` clauses at runtime. Also declares `findByIdAndUserId` — the single method that enforces per-user ownership at the database query level for every read/update/delete.

TaskSpecifications

A static utility class producing reusable `Specification<Task>` predicates: `ownedBy(userId)`, `matchesKeyword(keyword)` (case-insensitive `LIKE` over title OR description), `hasStatus(status)`, `hasPriority(priority)`. These are combined with `.and()` in the service, so the final SQL `WHERE` clause only ever contains the filters actually supplied by the caller.

TaskService

The business core of the task feature: `search()` builds the specification chain and a safety-checked `Pageable` (sort field whitelisted against a hardcoded `SORTABLE_FIELDS` set to block property-path injection; page size clamped to 1–100). `getOwnedTask()` is the single choke point every read/update/delete passes through, throwing `ResourceNotFoundException` if the task doesn't belong to the caller — meaning a 404 (not a 403) is returned for another user's task, so existence isn't leaked either. `create()` / `update()` / `delete()` are all `@Transactional`.

TaskController

Exposes `GET/POST /api/tasks`, `GET/PUT/DELETE /api/tasks/{id}`. Every method takes `@AuthenticationPrincipal User user` as its first parameter — the authenticated owner is always implicit, never taken from the request body or path, closing off any possibility of operating on another user's data via a forged payload.

DTOs — `TaskRequest`, `TaskResponse`, `PageResponse<T>`

`TaskRequest` validates `title` (required, ≤ 150 chars), `description` (≤ 5000 chars), and requires non-null `priority` / `status` enums. `PageResponse<T>` is a framework-agnostic pagination envelope (`content` , `page` , `size` , `totalElements` , `totalPages` , `first` , `last`) that wraps Spring Data's `Page<E>` without leaking Spring types into the JSON contract.

Package: `exception`

GlobalExceptionHandler `@RestControllerAdvice`

Six handlers, each mapping to one HTTP status: `MethodArgumentNotValidException` $\rightarrow$ 400 (with a per-field error list), `MethodArgumentTypeMismatchException` $\rightarrow$ 400, `ResourceNotFoundException` $\rightarrow$ 404, `EmailAlreadyExistsException` $\rightarrow$ 409, `BadCredentialsException` $\rightarrow$ 401, `AccessDeniedException` $\rightarrow$ 403, and a catch-all `Exception` $\rightarrow$ 500 with a generic message (so internal error details are never leaked to the client).

ApiError / FieldValidationError

The single record shape every error response conforms to: `timestamp` , `status` , `error` , `message` , `path` , and an optional `fieldErrors` list (only present for validation failures, thanks to `@JsonInclude(NON_NULL)`).

Frontend classes

AuthService

Owns the client-side session:

`login()` / `register()` POST to the backend and, via an RxJS `tap` , persist `{token, user}` to `localStorage` . Exposes a signal `user` and a derived `isAuthenticated` computed signal that the whole app (navbar, guards, root component) reacts to. `logout()` clears storage and navigates to `/login` .

TaskService

A thin `HttpClient` wrapper for `/api/tasks` — builds `HttpParams` for search/filter/sort/pagination and exposes `list/get/create/update/delete` as observables. Contains no state of its own.

authGuard / guestGuard

Functional `CanActivateFn` s. `authGuard` redirects unauthenticated users away from `/dashboard` to `/login` ; `guestGuard` does the inverse for `/login` and `/register` , keeping signed-in users on `/dashboard` .

authInterceptor / errorInterceptor

`authInterceptor` attaches `Authorization: Bearer <token>` to every outgoing request when a token exists. `errorInterceptor` centralizes failure handling: on a 401 from anywhere *except* an `/auth/` call, it logs the user out; for every error it normalizes the backend's `ApiError` body into a plain `Error` with a human-readable message the UI can display directly.

DashboardComponent

TaskFormComponent

The main authenticated screen. Holds all list state as signals (`tasks` , `loading` , `error` , `page` , `search` , `statusFilter` , `priorityFilter` , `sortValue`) and calls `TaskService.list()` on every change (debounced 300ms for free-text search). Opens `TaskFormComponent` as a modal for create/edit and confirms before delete.

Create/edit modal using Angular's new `input()` / `output()` signal APIs instead of `@Input()` / `@Output()` decorators. Pre-fills from the `task` input when editing; emits a validated `TaskRequest` via the `saved` output. Closes on Escape (`@HostListener`) or backdrop click.

SECTION 6

How the Frontend, Backend, and Database Communicate

All communication is synchronous request/response HTTP+JSON — there are no WebSockets, Server-Sent Events, or polling anywhere in the codebase.

1 **Angular component** → AuthService / TaskService

A component (e.g. `DashboardComponent`) calls a method on an injected service, which builds an `HttpClient` call (GET/POST/PUT/DELETE) against `environment.apiUrl` (`http://localhost:8080/api`).

2 **authInterceptor**

Attaches `Authorization: Bearer <token>` from `localStorage` (via `AuthService.getToken()`) if a token is present.

3 **Browser** → network (CORS)

Since the SPA (4200) and API (8080) are different origins, the browser applies CORS.

`SecurityConfig.corsConfigurationSource()` explicitly allows the configured origin(s), methods GET/POST/PUT/PATCH/DELETE/OPTIONS, all headers, and exposes the `Authorization` header back to the client.

4 **Spring servlet filter chain** → JwtAuthenticationFilter

Runs before any controller. Validates the bearer token and — if valid — populates

`SecurityContextHolder` with the authenticated `User`.

5 **Controller** → Service → Repository

The matched `@RestController` method delegates to a `@Service`, which uses a Spring Data repository to talk to the database via Hibernate-generated SQL over JDBC.

6 **PostgreSQL** → Hibernate → entities

Result rows are hydrated into JPA entities (`User`, `Task`), mapped to response DTOs, and serialized to JSON by Jackson inside the `ResponseEntity`.

7 **errorInterceptor** ← HTTP response

On success, the JSON flows straight back into the calling service's `Observable`. On failure, this interceptor first normalizes the backend's `ApiError` body into a friendly `Error` (and forces logout on an unexpected 401) before it reaches the component.

8 **Component** → signals → template

The component's `subscribe({ next, error })` handler updates its signals; Angular's change detection re-renders the template reactively.

SECTION 7

Authentication Flow (JWT)

Authentication is **stateless**: `SecurityConfig` sets `SessionCreationPolicy.STATELESS`, so no server-side session or cookie is ever created. Identity is carried entirely in a signed JWT that the client stores and re-sends on every request. There is no refresh-token mechanism in this codebase — the token is valid for a fixed lifetime (24h by default) and, once expired, the user must log in again.

7.1 Registration

1

RegisterComponent

Submits `{ name, email, password }` to `POST /api/auth/register` — permitted without a token (`SecurityConfig` whitelists `/api/auth/**`).

2

AuthService.register()

`userRepository.existsByEmail()` — if the email is taken, throws `EmailAlreadyExistsException` (→ **409 Conflict** via `GlobalExceptionHandler`).

3

Password hashing

The plaintext password is hashed with `BCryptPasswordEncoder` before a `User` (role `ROLE_USER`) is built and saved via `UserRepository`.

4

JwtService.generateToken()

Issues a JWT: subject = email, `issuedAt` = now, `expiration` = now + `app.jwt.expiration-ms`, signed with the HMAC key derived from `app.jwt.secret`.

5

Response → AuthService.storeSession()

`AuthResponse{token, "Bearer", user}` returns with **201 Created**. The frontend persists token + user to `localStorage`, sets the `user` signal, and the router navigates to `/dashboard`.

7.2 Login

1 LoginComponent

Submits `{ email, password }` to `POST /api/auth/login`.

2 AuthService.login()

Builds a `UsernamePasswordAuthenticationToken` and calls `AuthenticationManager.authenticate()`.

3 DaoAuthenticationProvider → AppUserDetailsService

Loads the `User` by email via `UserRepository.findByEmail`, then compares the submitted password against the stored BCrypt hash using the same `PasswordEncoder` bean.

✗ On mismatch

Spring Security throws `BadCredentialsException` → `GlobalExceptionHandler` → **401** `{"message": "Invalid email or password"}`.

4 On success → JwtService.generateToken()

A new JWT is issued for the authenticated principal and returned as an `AuthResponse`; the frontend stores the session exactly as in registration.

7.3 Authenticated request (e.g. loading the dashboard)

1 authInterceptor

Reads the stored token and adds `Authorization: Bearer <token>` to the outgoing request.

2 JwtAuthenticationFilter

Extracts the token, calls `jwtService.extractUsername()`, loads `UserDetails` via `AppUserDetailsService`, then `isTokenValid()` checks subject match + non-expiry.

3 SecurityContextHolder

If valid, a `UsernamePasswordAuthenticationToken` carrying the full `User` entity (not just a username string) is set as the current `Authentication` — this is exactly what later lets `@AuthenticationPrincipal User user` resolve directly in controllers.

✗ Missing / invalid / expired token

The filter never throws — it silently leaves the request unauthenticated (or clears the context on a parse error). `SecurityConfig`'s `.anyRequest().authenticated()` then rejects the request, and `JwtAuthenticationEntryPoint` writes a uniform **401** `ApiError` JSON body.

4 errorInterceptor (frontend)

On a 401 from any non- `/auth/` call, calls `auth.logout()` — clearing storage and redirecting to `/login` — so an expired token self-heals into a clean re-login rather than a stuck error state.

Notable gap: `Role.ROLE_ADMIN` exists in the data model but is never enforced — every authenticated user has identical permissions. There is also no logout endpoint or token blacklist on the backend; "logout" is purely a client-side action (clearing `localStorage`). Because tokens are stateless and self-contained, a token remains cryptographically valid until it naturally expires, even after a client-side logout.

SECTION 8

Database Structure

The schema has exactly two tables, defined by the `User` and `Task` `@Entity` classes and mirrored in `database/schema.sql`. In development, Hibernate creates and updates these tables automatically (`ddl-auto: update`); the SQL file is the equivalent hand-written DDL intended for `ddl-auto: validate` in production.

users	
id	BIGINT, PK, identity
name	VARCHAR(100) NOT NULL
email	VARCHAR(150) NOT NULL, UNIQUE
password	VARCHAR(255) NOT NULL (BCrypt)
role	VARCHAR(20) DEFAULT 'ROLE_USER'
created_at	TIMESTAMPTZ NOT NULL

1 — *

fk_tasks_user
ON DELETE
CASCADE

tasks	
id	BIGINT, PK, identity
title	VARCHAR(150) NOT NULL
description	TEXT, nullable
priority	VARCHAR(20) DEFAULT 'MEDIUM'
status	VARCHAR(20) DEFAULT 'TODO'
due_date	DATE, nullable
user_id	BIGINT NOT NULL, FK → users.id
created_at	TIMESTAMPTZ NOT NULL
updated_at	TIMESTAMPTZ NOT NULL

One `users` row owns many `tasks` rows. Deleting a user cascades to delete all of their tasks (`ON DELETE CASCADE`).

Indexes

Index	Column	Purpose
uk_users_email	users.email	Enforces one account per email; backs <code>existsByEmail</code> / <code>findByEmail</code> lookups
idx_tasks_user	tasks.user_id	Every task query is scoped by owner (<code>TaskSpecifications.ownedBy</code>) — this index is on the hot path of every request
idx_tasks_status	tasks.status	Backs the status filter
idx_tasks_priority	tasks.priority	Backs the priority filter
idx_tasks_due_date	tasks.due_date	Backs sorting/filtering by due date

Mapping notes

- `User` and `Task` both use `GenerationType.IDENTITY` primary keys, matching PostgreSQL's `GENERATED BY DEFAULT AS IDENTITY`.
- `Task.user` is `@ManyToOne(fetch = FetchType.LAZY, optional = false)` — the owner is never optional, and the association is not eagerly joined unless explicitly navigated.

- `priority / status / role` are persisted as their enum **name** (`@Enumerated(EnumType.STRING)`), so the stored values are human-readable strings (`"HIGH"` , `"TODO"`) rather than ordinals.
- `open-in-view: false` is set in `application.yml` — there is no lazy-loading-in-the-view-layer trap; all entity access happens inside the `@Transactional` service methods, before DTO projection.

API Request Flow

Client → Controller → Service → Repository → Database → Response, traced through a concrete example:

```
GET /api/tasks?search=report&status=TODO&priority=HIGH&sortBy=dueDate&direction=asc&page=0&size=10
```

- 1 **DashboardComponent** → `TaskService.list()`
Builds an `HttpParams` object from the current search/filter/sort/page signals and issues an `HttpClient.get`.
- 2 **authInterceptor**
Adds the bearer token header.
- 3 **JwtAuthenticationFilter**
Authenticates the request and sets `SecurityContextHolder` before the request reaches `DispatcherServlet` routing.
- 4 **TaskController.list()**
Resolves `@AuthenticationPrincipal User user` and binds query params (`search` , `status` , `priority` , `page` , `size` , `sortBy` , `direction` — with controller-level defaults of `page=0` , `size=10` , `sortBy=createdAt` , `direction=desc` when the frontend doesn't specify them). Calls `taskService.search(...)`.
- 5 **TaskService.search()**
Starts from `TaskSpecifications.ownedBy(user.getId())` — always applied, regardless of what the caller asks for — then conditionally ANDs `matchesKeyword` , `hasStatus` , `hasPriority` only for the parameters actually supplied. Builds a safe `Pageable` : `sortBy` is checked against the `SORTABLE_FIELDS` whitelist (falls back to `createdAt` if not recognized) and `size` is clamped to [1, 100].
- 6 **TaskRepository** (`JpaSpecificationExecutor`)
Hibernate translates the composed `Specification` + `Pageable` into one parameterized `SELECT` (with a `WHERE user_id = ? AND (lower(title) LIKE ? OR lower(description) LIKE ?) AND status = ? AND priority = ? ORDER BY due_date ASC LIMIT ? OFFSET ?`) plus a matching `COUNT` query for `totalElements` , executed against **PostgreSQL** over JDBC.
- 7 **Result** → `Page<Task>` → `PageResponse<TaskResponse>`
Back in the controller, `PageResponse.from(result, TaskResponse::from)` projects each `Task` entity into a `TaskResponse` DTO and wraps the page metadata into the framework-agnostic envelope.
- 8 **Response**
`ResponseEntity.ok(pageResponse)` is serialized to JSON by Jackson and sent back over HTTP.
- 9 **Angular** → signals → template
`DashboardComponent` 's subscription updates `tasks` , `totalElements` , and `totalPages` signals; the grid re-renders.

Write paths (create / update / delete)

The same layering applies to mutations, with one addition: **Bean Validation** runs before the controller method body executes. `@Valid @RequestBody TaskRequest request` on `create / update` triggers `MethodArgumentNotValidException` on any constraint violation (blank title, oversized description, null priority/status) — caught by `GlobalExceptionHandler` and returned as **400** with a per-field `fieldErrors` array before `TaskService` is ever invoked. For `update / delete`, `TaskService.getOwnedTask()` is the ownership gate: it re-fetches by `(id, user.getId())`, so attempting to modify another user's task returns **404 Not Found** — the same response as a genuinely nonexistent id, which avoids revealing whether the id belongs to someone else.



Any exception at any point (validation, not-found, DB constraint) is intercepted by `GlobalExceptionHandler` and rendered as a uniform `ApiError` before it reaches the client.

SECTION 10

API Reference

Base path `/api`. Every route except `/api/auth/**` and `/actuator/health` requires `Authorization: Bearer <token>`.

Method	Endpoint	Auth	Body	Description
POST	<code>/api/auth/register</code>	Public	<code>{name, email, password}</code>	Create account, returns JWT + user
POST	<code>/api/auth/login</code>	Public	<code>{email, password}</code>	Sign in, returns JWT + user
GET	<code>/api/users/me</code>	Bearer	—	Current authenticated user
GET	<code>/api/tasks</code>	Bearer	query params	List own tasks, paged & filterable
GET	<code>/api/tasks/{id}</code>	Bearer	—	Get one owned task (404 if not owner)
POST	<code>/api/tasks</code>	Bearer	<code>TaskRequest</code>	Create a task for the caller
PUT	<code>/api/tasks/{id}</code>	Bearer	<code>TaskRequest</code>	Full update of an owned task
DELETE	<code>/api/tasks/{id}</code>	Bearer	—	Delete an owned task (204)

`GET /api/tasks` query parameters

Param	Type	Default	Notes
search	string	—	Case-insensitive match on title OR description
status	TODO IN_PROGRESS DONE	—	Exact match
priority	LOW MEDIUM HIGH	—	Exact match
page	int	0	Clamped to ≥ 0
size	int	10 (backend) / 9 (frontend default)	Clamped to 1–100
sortBy	string	createdAt	Whitelisted: createdAt, updatedAt, dueDate, priority, status, title
direction	asc desc	desc	Case-insensitive

`TaskRequest` shape

```
{
  "title": "Write project README",
  "description": "Cover setup + API",
  "priority": "HIGH",
  "status": "IN_PROGRESS",
  "dueDate": "2026-07-15"
}
```

Error shape (`ApiError`) — returned for every non-2xx response

```
{
  "timestamp": "2026-07-15T10:22:31.442Z",
  "status": 400,
  "error": "Validation Failed",
  "message": "One or more fields are invalid",
  "path": "/api/tasks",
  "fieldErrors": [
    { "field": "title", "message": "Title is required" }
  ]
}
```

This document was produced by direct inspection of the source in `task-management-system/backend`, `/frontend`, and `/database` — no functionality has been assumed or invented beyond what the code implements.